Phase 2 | Complete Source-Code Deep Dive

Follow the actual execution path and inspect critical modules.

FastAPI Edition | Repository-grounded study material | Snapshot: 2026-10-24

1. Repository reading order

Start with `run.py`, `app/__init__.py`, `app/routes.py`; then `app/forecasting.py`, `app/calendar_context.py`, `app/operating_hours.py`, `app/scheduling_optimizer.py`, `app/travel_graph.py`, and `app/network_topology.py`. Next inspect `pipeline/01_audit_and_clean_data.py`, `pipeline/02_generate_calibrated_dataset.py`, `pipeline/03_train_and_evaluate_forecaster.py`, `models/`, `data/cleaned/`, and templates/static JavaScript. Read `requirements.txt` and `Procfile` last to connect code to deployment.

2. Entry points and startup

`run.py` exposes `app` and can start Uvicorn when executed directly. Render's Procfile uses `uvicorn run:app --host 0.0.0.0 --port $PORT`. Importing the route module reads station CSV and builds the graph at module scope. Importing forecasting loads joblib model/metadata and station data. Scheduling loads operations CSV and optional line split JSON. This is easy to reason about for a single process but creates startup coupling to local artifacts.

Interview trace: import failure in model loading means application import fails and server never begins serving. The route graph also stays in memory for the lifetime of each worker. A future lifespan handler could validate and load dependencies explicitly, surface clearer startup errors, and support test fixtures.

3. Route contracts

- `GET /`: landing template and scenario preview.
- `GET /dashboard`: dashboard template and station metadata.
- `POST /get_footfall`: form input, HTML result; uses `request.form()` and `run_in_threadpool`.
- `GET/POST /api/predict`: query parameters or JSON object; returns predictions, scheduling, travel times, and provenance labels.
- `GET /api/stations`: station metadata JSON.
- `GET /api/model_metrics`: evaluation summary.
- `/api/docs`: FastAPI-generated OpenAPI docs.

`api_predict` rejects malformed JSON and non-object JSON with 400, then passes input to `_predict_data`. `_parse_datetime` validates date/time syntax and returns a user-readable 400 response. There is not yet a fully typed Pydantic request model, consistent unknown-station 404 behavior, authentication, or rate limiting. A syntactically valid but unknown station can fall through to default metadata and produce misleading features; validate station membership before inference.

4. Forecasting walkthrough

At module import, `forecasting.py` loads `_MODEL`, `_META`, and `_STATIONS_DF`. `get_station_info` retrieves line/interchange/coordinate fields or a fallback record. `predict_hourly_footfall` builds the 23 features: calendar parts, weekend/holiday/workday flags, cyclic sin/cos for hour/day/month, station/line/typology indices, interchange indicators and training-derived means. It selects columns in `_META['feature_cols']` order, calls `.predict()`, clips negative output to zero and rounds to integer.

Feature schema alignment is a contract: feature name, order, type, category mapping, and missing-value handling must stay compatible with the saved estimator. Unknown station fallback currently maps index zero; that avoids an exception but may silently treat an unknown station like a known station. Better behavior is validation plus clear 404/422 response.

`predict_footfall_with_minute_interpolation` returns zero outside service hours; otherwise interpolates between the selected hour and the next hour by minute fraction. This is display smoothing, not a minute-level trained model. `predict_station_full_day_profile` evaluates hours 5 through 23, creates 19 points, obtains peak and total, and apportions interchange curves using configured splits.

5. API request flow

`_predict_data` parses station/date/time/line. It derives date context and service-open state; calls instantaneous, hourly, and full-day predictions; calls `optimize_station_schedule`; optionally selects a line or aggregates all lines for interchange station summary; overrides the schedule with a closed state when service is off; runs shortest-path graph search; then serializes a JSON response including provenance.

The async route delegates synchronous Pandas/sklearn work to Starlette's threadpool. This keeps the event loop available, but doesn't make inference faster and doesn't provide unlimited concurrency. Under load, threadpool saturation and memory per worker matter.

6. Schedule logic walkthrough

`get_interchange_line_split` uses configured proportions or equal-share fallback. `optimize_station_schedule` allocates total station demand across lines, then calls `optimize_line_schedule` per line. That function reads capacity, fleet, headway and turnaround values; estimates fleet/signaling bounds; chooses peak/off-peak baseline; estimates boarding clearance per train based on a through-load factor; compares demand with existing clearance; chooses increase/maintain/decrease action; clamps frequency; rounds to a practical interval; computes utilization, wait estimate, and risk badge.

Key distinctions: nominal train capacity is not the same as boarding clearance; clearance reserves capacity for passengers already onboard. The 65%/40% through-load assumptions are model assumptions. `capacity_utilization_pct` describes estimated boarding demand divided by modeled offered clearance per hour. It does not estimate train-by-train load or alighting.

7. Calendar and operating hours

`calendar_context.py` maps a date into workday/weekend/occasion categories according to project calendar data. `operating_hours.py` decides whether a time lies within configured modeled service hours; current route text specifies 05:00-23:30. The training curve has hourly labels from 05:00 through 23:00. Explain the difference between a model time slot and the service-window gate.

8. Mapping and graph code walkthrough

- `_load_stations` reads canonical station labels and coordinate strings from CSV.
- `_normalize` strips selected footnote marks, normalizes case and dash variants, removes punctuation and reduces duplicate spaces.
- `_match_topology_to_canonical` gathers all names from ordered segments and interchange keys, matches normalized names, then tries `ALIASES`.
- `build_graph` counts repeated coordinates and treats missing or common repeated coordinate pairs as placeholders.
- For each pair of consecutive route stops, it maps both endpoints; unmatched endpoints cause that edge to be skipped.
- If coordinates are plausible, haversine distance / average speed gives an edge duration with a 1-minute floor; if duration exceeds 8 minutes, use 2.5-minute default. Missing/placeholder coordinates also use that default.

- Edge is added both ways for this prototype.
- Interchange map adds bidirectional same-station line-node edges with a 5-minute penalty.
- Dijkstra starts from all line nodes of the source, settles via heap, relaxes neighbors, then takes the minimum line-node distance per physical station and removes source.

Potential edge cases: Unicode punctuation/alias mismatch; duplicate canonical normalized names; topology name silently unmatched; stations with no graph node; branch alias; repeated coordinates incorrectly flagged; directed/service-specific links modeled as bidirectional; transfer cost varies by station but is constant here; source name mismatch returns empty map.

9. Offline ML pipeline

Step 1 audits/cleans station/demand data. Step 2 generates calibrated synthetic hourly data from station/date/hour and scenario rules. Step 3 joins station attributes, derives date/time/cyclic features, encodes station/line/typology, splits chronologically at 2026-10-01, computes historical aggregates without using future holdout rows, trains baselines and HistGradientBoostingRegressor, evaluates, and saves model/metadata/metrics.

Leakage defense: split is chronological, training historical features use prior records/dates, and holdout features use aggregates built only on training rows. Ask whether any feature could contain target information; describe how to inspect feature construction and split logic. A synthetic generator can still encode patterns that the model learns very well; leakage prevention does not transform synthetic results into real-world validation.

10. Practical code exercises

- Trace one request for a closed hour and list which functions return zero/closed status.
- Change a station to an unknown name and observe the current API behavior; propose a validation response.
- Print every topology label that fails canonical matching.
- Trace Rajiv Chowk -> transfer -> destination on line-expanded nodes.
- Remove one interchange edge in a test graph and explain route changes.
- Verify model input column order matches saved metadata.
- Compare output before/after one feature value changes while keeping date and station constant.

How to use this series

Read Phase 1 first, then Phase 2 while opening the source files. Use Phases 3, 5, 6, and 7 to deepen technical understanding; rehearse with Phases 4 and 8; revise with Phase 9. The editable Markdown beside each PDF is the source of truth for corrections.

Evidence and honesty rules

MetroInsights is a planning prototype. Its demand targets and holdout are synthetic. Model metrics describe agreement with generated scenario labels, not real passenger counts or operational DMRC accuracy. Network topology, coordinates, capacities, headways, service frequency, and interchange splits are project data or configurable assumptions unless separately verified. Say “estimated” and explain the assumptions. Do not describe this as a live passenger-count, train occupancy, or production DMRC system.

Verified snapshot used in this edition

The cleaned station table contains 243 station records. The topology graph's destination count depends on the selected source and excludes the source itself; 242 destinations can be returned when all other modeled stations are connected. Do not say 226 or 228 without checking the current data. The project defines 11 route keys, including branch keys. The synthetic training series uses 243 stations x 728 dates x 19 hourly slots (05:00-23:00), or 3,361,176 rows. A chronological holdout begins 2026-10-01 and has 438,615 rows. The recorded HistGradientBoostingRegressor scores are MAE 26.58, RMSE 51.10, R2 0.9946, and MAPE 5.18%; Ridge MAE is 53.67 and historical station/day/hour mean MAE is 60.25. These are synthetic-only evaluation results.

Current stack and constraints

FastAPI + Uvicorn, Starlette/Jinja2, server-rendered HTML with JavaScript/CSS, Pandas, NumPy, scikit-learn, joblib, CSV/JSON, and a custom adjacency graph with heapq Dijkstra. No relational database, live ridership feed, authentication, or distributed job queue is present. The generated multi-million-row CSV is a training input; runtime inference loads compact serialized model/metadata plus station and operations inputs, rather than reading every training row on each request. Runtime startup imports routes and eagerly loads artifacts/builds the graph, so missing files can prevent startup. The current FastAPI migration has passed static compilation and diff checks in prior work, but an actual runtime smoke test was not possible in the offline environment; verify locally before claiming deployment verification.